stores information about neighbours adjacent to it in the *nodeId* space. It also notifies the application about the arrival and departure of nodes in the adjacent *nodeId* space.

Each node is assigned a *nodeId*, which is the md5 hash value of its IPv4 address plus port. The *nodeId* specifies the position of the node in a circular *nodeId* space. In a network on *N* nodes it can take up to *ceil(log2^bN)* steps for a message to reach its destination node.

We will look at the routing procedure and self-organisation in detail after a walk-through of the node state.

The application's architecture has the following key classes:

- *Database* - It follows a singleton design pattern to store all the information relevant to a node like routing tables, neighbourhood sets, etc. The *C++ STL* (standard template library) provides locks used for mutual exclusion, implementing fine-grained locking. *nodeId* related information is stored inside the Node class object.
- *LogHandler* - A singleton class, which is used to write all the logs — from the running thread to the log files (*LogError/LogMsg*).
- *Printer* - This class is specified to display informative messages on the console.
- *NetworkInterface* - This class performs read/write operations from the network, i.e., an interface between the application and network, using UNIX provided network system calls.

Figure 2 depicts the architecture of each node in the application. There are four key classes and separate handlers for user commands and communication between nodes in the application's network. Command Listener handles user requests, whereas Peer Handler listens for the messages that define the contracts for communication between nodes.

Besides the key classes, Command Handler listens to the user requests and creates a new thread for the corresponding action to be taken, whereas the Peer Handler listens for requests from other nodes in the network and calls for the corresponding method as a new thread. A few sets of messages have been specified that determine which remote procedure will get invoked. Hence, these messages serve as API endpoints for our application.

The implementation uses Protocol Buffers for message serialisation (encoding) and deserialisation (decoding). Listed below are the messages used to define the interface in the application:

- *JoinMe* - Upon the arrival of a new node in the network, send this to all adjacent nodes to update their network information.
- *Join* - Like any other message, this message gets routed by the node that receives the new node's joining request.
- *RoutingUpdate* - Existing nodes send this message to the newly joined ones. This information is used by newly joined nodes to set up their routing tables and other network information.
- *AllStateUpdate* - Propagate updates to all entries of nodes in the current node's routing table.
- *AddToHashTable* - To add key-value pairs to the present node.
- *SetValue* - Store key-value pair.
- *GetValue* - Get the value of the given key, and route it to the source that requested the value.
- *DeleteNode* - Remove an entry for a node from the routing tables.
- *Shutdown* - Shutdown the entire network, making all nodes exit.

Let us go through the details of the node to understand the working procedure of the application's routing of messages and the self-organisation used for responding to the arrival, failure and departure of nodes in the network.

Figure 3 depicts the state of an application's node, listing the critical information required by a node.

Each node maintains a leaf set, a neighbourhood set and a routing table. A node has to have a routing

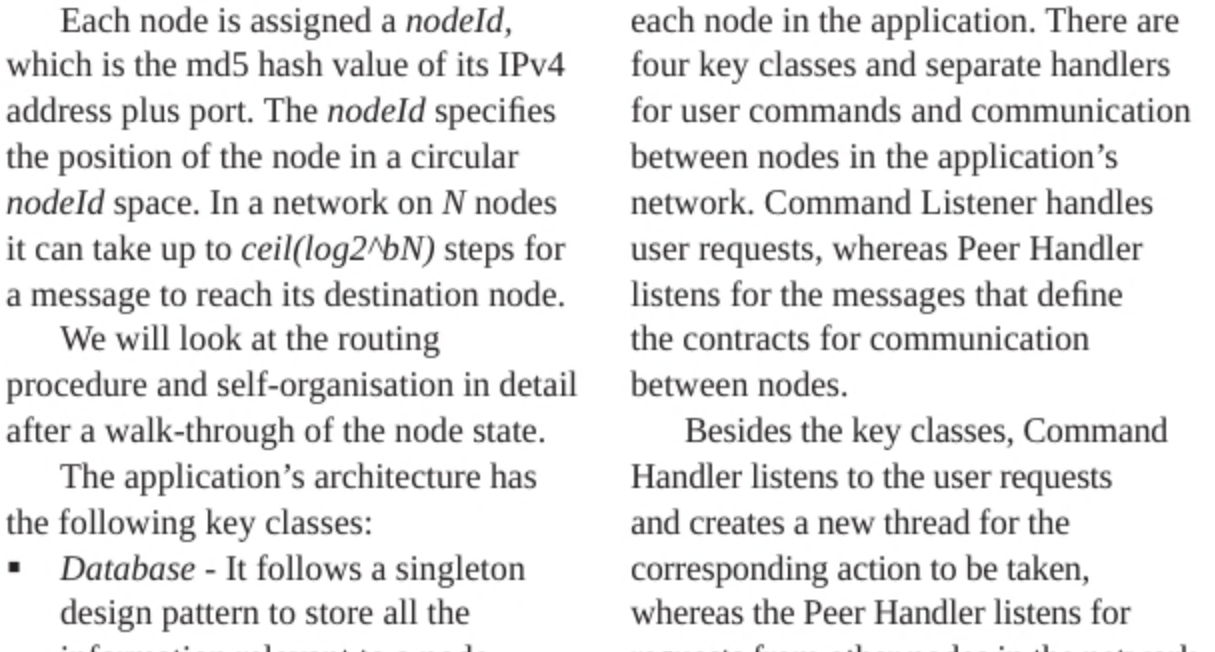

Figure 2: Architecture of the application's node

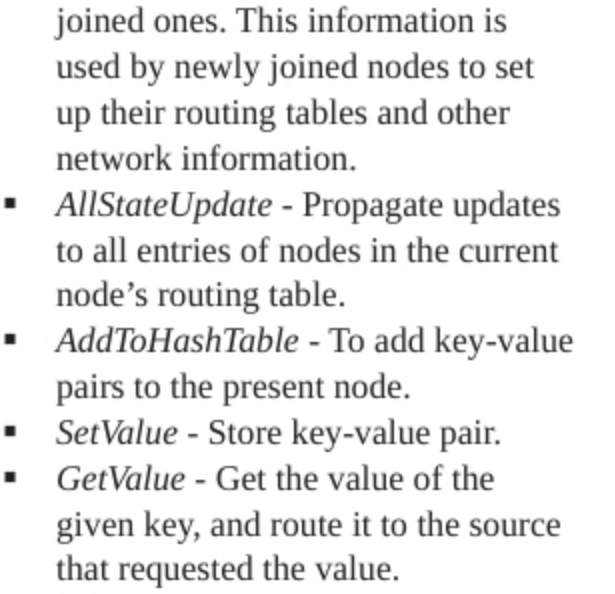

Figure 3: State of an application's node